# GIT & GITHUB

Guide pratique
Fondamentaux, branches, push et merge

Un support de référence pour apprendre à gérer un dépôt local, publier son code sur GitHub, travailler avec des branches et intégrer des modifications proprement.

Objectif : comprendre le rôle de chaque commande et savoir dans quel ordre l'utiliser dans un projet réel.

## Sommaire


0. Initialiser et publier un dépôt Git local
1. Le cycle quotidien : status, diff, add et commit
2. Consulter l'historique des commits
3. Comprendre et gérer les branches
4. Publier une branche sur GitHub
5. Intégrer une branche : Pull Request ou merge local
6. Synchroniser son dépôt local avec GitHub
7. Commandes essentielles et bonnes pratiques

# 0. Initialiser et publier un dépôt Git local

Cette première partie décrit le parcours de base : transformer un dossier en dépôt Git, enregistrer un premier commit, connecter le dépôt à GitHub et publier la branche principale.

## Étape 0 — Initialiser un dépôt local

```
git init
```

Initialise Git dans le dossier courant. Git crée un répertoire caché `.git` qui contient les informations nécessaires au suivi du projet.

Avant de commencer, place-toi dans le dossier de ton projet dans le terminal de VS Code.

## Étape 1 — Vérifier l'état du dépôt

```
git status
```

Affiche les fichiers nouveaux, modifiés, préparés pour le commit ou déjà enregistrés. C'est une commande de vérification à utiliser très régulièrement.

## Étape 2 — Examiner les modifications

```
git diff
```

Affiche les différences entre les fichiers modifiés dans le répertoire de travail et la dernière version enregistrée. Les lignes ajoutées sont généralement précédées de + et les lignes supprimées de `-`.

## Étape 3 — Préparer les modifications

```
git add .
```

Ajoute à la zone de préparation (staging area) les modifications et fichiers concernés dans le dossier courant. Le point `.` signifie que Git prend en compte les changements situés dans le répertoire courant et ses sous-dossiers.

Conseil : si tu veux préparer un seul fichier, utilise plutôt `git add nom-du-fichier`. Cela évite d'inclure accidentellement d'autres modifications.

## Étape 4 — Vérifier ce qui est préparé

```
git status
```

Vérifie que les bons fichiers apparaissent dans la section `Changes to be committed`. Ces changements sont prêts à être enregistrés.

## Étape 5 — Enregistrer les modifications

```
git commit -m "Your commit message"
```

Crée un commit, c'est-à-dire un instantané des modifications préparées. Le message doit décrire clairement le changement réalisé, par exemple `Add contact form` ou `Fix login validation`.

## Étape 6 — Vérifier l'historique

```
git log
# Version condensée :
git log --oneline
```

Affiche les commits précédents. La version `--oneline` présente chaque commit sur une seule ligne, ce qui facilite la lecture de l'historique.

## Étape 7 — Connecter le dépôt local à GitHub

```
git remote add origin <url_de_votre_depot>
```

Associe le dépôt local à un dépôt distant sur GitHub. `origin` est le nom conventionnel donné à ce dépôt distant.

Note : si le dépôt n'existe pas encore sur GitHub, il faut d'abord le créer sur github.com, puis copier son URL HTTPS et l'utiliser dans cette commande. Si le dépôt distant est déjà configuré, ne répète pas cette commande.

**Pour vérifier l'adresse du dépôt distant :**

```
git remote -v
```

## Étape 8 — Renommer la branche principale si nécessaire

```
git branch -M main
```

Renomme la branche locale actuelle en `main`. Cette commande est utile si ton dépôt a été initialisé avec un nom de branche différent, comme `master`.

## Étape 9 — Envoyer la branche principale sur GitHub

```
git push -u origin main
```

Envoie les commits de la branche locale `main` vers le dépôt distant `origin`. Le paramètre `-u` configure la branche distante suivie (upstream). Après cette première publication, un simple `git push` suffit généralement.

## Étape 10 — Récupérer les modifications depuis GitHub

```
git pull origin main
```

Récupère les nouveaux commits de `origin/main` et les intègre dans la branche locale actuellement active. Pour mettre à jour `main`, il faut donc être positionné sur `main` avant d'exécuter cette commande.

À retenir : `git push` envoie des commits vers le dépôt distant ; `git pull` récupère les commits distants et les intègre à la branche locale active.

# 1. Le cycle quotidien : status, diff, add et commit

Lorsqu'un fichier est modifié dans VS Code, voici un ordre de travail recommandé :

```
# 1. Vérifier l'état des fichiers
git status

# 2. Examiner les modifications non préparées
git diff

# 3. Préparer les modifications
git add .

# 4. Vérifier ce qui est préparé
git status

# 5. Enregistrer les modifications
git commit -m "Describe your changes"
```

Les commandes `git status` et `git diff` sont des commandes de vérification : elles ne sont pas obligatoires pour créer un commit, mais elles sont vivement recommandées, surtout pendant l'apprentissage et dans un projet partagé.

Pour examiner les modifications déjà préparées dans la staging area, utilise :

```
git diff --staged
```

### Les trois états à comprendre :

| État | Signification |
|---|---|
| Working directory | Les fichiers ont été modifiés dans le dossier de travail. |
| Staging area | Les changements ont été sélectionnés avec git add et sont prêts à être commités. |
| Repository / commit | Les changements ont été enregistrés dans l'historique Git. |

# 2. Consulter l'historique des commits

```
git log
git log --oneline
```

Utilise `git log` pour consulter l'historique détaillé, notamment les identifiants, auteurs, dates et messages des commits. Utilise `git log --oneline` pour une vue condensée.

# 3. Comprendre et gérer les branches

Une branche permet de développer une fonctionnalité, corriger un bug ou expérimenter sans modifier directement la branche principale. Dans beaucoup de projets, `main` contient la version principale du code, tandis que les développeurs travaillent sur des branches secondaires.

### Étape 1 — Vérifier la branche actuelle

```
git status
git branch
```

La commande `git branch` affiche les branches locales. L'astérisque * indique la branche active.

### Étape 2 — Créer une branche

```
git branch feature/add-new-messages
```

Crée une branche locale appelée `feature/add-new-messages`. La commande ne change pas automatiquement de branche.

### Étape 3 — Changer de branche

```
git checkout feature/add-new-messages
# Alternative moderne :
git switch feature/add-new-messages
```

La commande `git checkout` permet notamment de changer de branche. `git switch` est une alternative plus récente, dédiée au changement de branche.

### Convention de nommage des branches

Le préfixe décrit le type de travail et le reste du nom précise l'objectif. Ces préfixes sont des conventions d'équipe, pas des mots-clés spéciaux de Git.

| Préfixe | Utilisation |
|---|---|
| feature/ | Développer une nouvelle fonctionnalité. |
| bugfix/ | Corriger un bug. |
| hotfix/ | Corriger rapidement un problème urgent. |
| docs/ | Modifier la documentation. |
| refactor/ | Améliorer la structure du code sans changer son comportement attendu. |

Attention : une branche créée localement n'apparaît pas automatiquement sur GitHub. Il faut la publier avec git push.

## 4. Publier une branche sur GitHub

Après avoir créé et sélectionné une branche, tu peux modifier tes fichiers, préparer les changements et créer un commit. Ensuite, publie la branche.

### Première publication de cette branche

```
git push -u origin feature/add-new-messages
```

Cette commande :

- envoie les commits de la branche locale vers GitHub ;
- crée la branche distante si elle n'existe pas encore ;
- configure l'upstream, c'est-à-dire la branche distante que la branche locale suivra.

Pour les publications suivantes, si l'upstream est configuré :

```
git push
```

Git sait alors vers quelle branche distante envoyer les commits. Pour voir les branches et leurs relations de suivi :

```
git branch -vv
```

### Pourquoi git push fonctionnait déjà sur main ?

Lors de la première publication de main, la commande `git push -u origin main` a configuré son upstream. Une nouvelle branche locale n'a pas nécessairement cet upstream : il faut donc faire une première publication avec `-u`. Après cela, `git push` fonctionne aussi sur cette branche.

Être positionné sur une branche ne suffit pas à la connecter à GitHub. Le suivi upstream doit être configuré.

# 5. Intégrer une branche : Pull Request ou merge local

Le merge consiste à intégrer dans une branche les commits provenant d'une autre branche. Par exemple, on peut intégrer le travail de `feature/add-new-messages` dans `main`.

## Méthode 1 — Pull Request sur GitHub (workflow collaboratif)

Une Pull Request (PR) est une demande d'intégration des modifications d'une branche dans une autre. Elle permet aux autres développeurs d'examiner le code, de commenter et de demander des corrections avant le merge.

- Publier la branche secondaire avec git push.
- Ouvrir le repository sur GitHub et aller dans Pull requests.
- Cliquer sur New pull request.
- Choisir Base : main et Compare : feature/add-new-messages.
- Créer la Pull Request et ajouter un titre et une description clairs.
- Faire examiner les changements (Code Review), si le projet le prévoit.
- Cliquer sur Merge pull request puis confirmer le merge.

| Champ | Branche | Rôle |
|---|---|---|
| Base | main | Branche qui reçoit les modifications. |
| Compare | feature/add-new-messages | Branche qui contient les modifications proposées. |

La Pull Request est une fonctionnalité de GitHub (et d'autres plateformes d'hébergement Git). Ce n'est pas une commande Git exécutée dans le terminal.

## Méthode 2 — Fusionner directement dans VS Code avec git merge

Cette méthode fusionne les branches localement, sans passer par une Pull Request. Elle est utile pour comprendre la commande merge et pour certains projets personnels.

## Étape 1 — Se placer sur la branche qui va recevoir les changements

```
git checkout main
```

La branche active doit être la branche cible. Ici, les modifications doivent être intégrées dans main.

## Étape 2 — Mettre à jour main depuis GitHub

```
git pull origin main
```

Récupère les dernières modifications distantes de main et les intègre dans la branche locale main. Cette étape aide à éviter de travailler sur une version locale dépassée.

### Étape 3 — Fusionner la branche secondaire

```
git merge feature/add-new-messages
```

Git intègre dans la branche actuelle (main) les commits de feature/add-new-messages. Il faut donc exécuter cette commande depuis main.

### Étape 4 — Envoyer main sur GitHub

```
git push origin main
```

Publie sur GitHub le résultat de la fusion locale.

Important : après avoir fusionné une Pull Request sur GitHub, ne refais pas le même merge localement. Mets simplement ta branche locale à jour avec git pull origin main.

## 6. Synchroniser son dépôt local avec GitHub

Si le merge a été réalisé sur GitHub via une Pull Request, mets à jour ton dépôt local ainsi :

```
git checkout main
git pull origin main
```

La première commande revient sur main. La seconde récupère et intègre dans main les changements qui ont été fusionnés sur GitHub.

Après le merge, la branche feature peut être conservée pour référence ou supprimée lorsqu'elle n'est plus nécessaire. Sur GitHub, une Pull Request fusionnée propose souvent de supprimer la branche source. En local, tu peux la supprimer après être revenu sur main :

```
git branch -d feature/add-new-messages
```

L'option `-d` supprime une branche locale déjà fusionnée. N'utilise pas une option de suppression forcée si tu n'es pas certain que son travail est sauvegardé ou fusionné.

# 7. Commandes essentielles et bonnes pratiques

| Commande | Fonction |
|---|---|
| git init | Initialiser un dépôt Git local. |
| git status | Vérifier l'état des fichiers. |
| git diff | Examiner les modifications non préparées. |
| git diff --staged | Examiner les modifications préparées. |
| git add . | Préparer les modifications du dossier courant. |
| git commit -m "message" | Enregistrer les modifications préparées. |
| git log --oneline | Afficher l'historique condensé. |
| git remote -v | Afficher les dépôts distants configurés. |
| git branch | Afficher les branches locales. |
| git branch nom | Créer une branche locale. |
| git checkout nom | Changer de branche. |
| git switch nom | Alternative moderne pour changer de branche. |
| git branch -vv | Afficher les branches et leur upstream. |
| git push -u origin nom | Publier une branche et configurer son upstream. |
| git push | Envoyer les commits vers la branche distante suivie. |
| git pull origin main | Récupérer et intégrer les commits distants de main dans la branche active. |
| git merge nom | Fusionner la branche indiquée dans la branche actuelle. |
| git branch -d nom | Supprimer une branche locale déjà fusionnée. |

## Bonnes pratiques à retenir

- Vérifier la branche active avec git branch avant de commencer à travailler.
- Éviter de développer directement sur main dans un projet collaboratif.
- Créer une branche dédiée pour chaque fonctionnalité ou correction.
- Faire des commits petits, cohérents et accompagnés de messages explicites.
- Examiner les changements avec git diff et vérifier la staging area avec git status.
- Publier la branche avec git push avant d'ouvrir une Pull Request.
- Privilégier les Pull Requests pour permettre la revue du code avant le merge.
- Synchroniser régulièrement main avec GitHub.

## Workflow complet à mémoriser

```
# 1. Mettre main à jour
git checkout main
git pull origin main

# 2. Créer et rejoindre une branche de travail
git branch feature/new-feature
git checkout feature/new-feature

# 3. Modifier les fichiers, puis préparer et enregistrer
git status
git diff
git add .
git status
git commit -m "Add new feature"

# 4. Publier la branche (première fois)
git push -u origin feature/new-feature

# 5. Ouvrir une Pull Request sur GitHub
# Base: main
# Compare: feature/new-feature
# Puis faire le merge après vérification.

# 6. Revenir sur main et récupérer le résultat
git checkout main
git pull origin main
```

Idée essentielle : une branche secondaire permet de travailler indépendamment de main. Une Pull Request propose l'intégration du travail ; le merge réalise cette intégration. Enfin, git pull synchronise le dépôt local avec les changements distants.